

# Capitolo 6 - Strumenti di sincronizzazione

Status Done

## Problema: Integrità dei dati condivisi

### Concorrenza vs. Parallelismo

- **Esecuzione concorrente:** un'unica CPU (core) passa rapidamente da un processo all'altro (tramite scheduling). Un processo viene interrotto (preemption) e ripreso più tardi.
- **Esecuzione parallela:** processi o thread diversi sono eseguiti contemporaneamente su core distinti di un sistema multicore.

Sia che l'esecuzione sia concorrente (intervallata) o parallela (simultanea), la problematica sorge quando i processi/thread condividono dati.

### Esempio Produttore-Consumatore

- **Produttore:** aggiunge elementi al buffer.
- **Consumatore:** preleva elementi dal buffer.
- **Variabile condivisa:** una variabile intera chiamata `counter`, che traccia il numero di elementi presenti nel buffer.
  - Il produttore esegue `counter++`.
  - Il consumatore esegue `counter--`.

### Condizione di gara (Race Condition)

L'errore si verifica perché un'operazione semplice come `counter++` (o `counter--`) non è un'azione singola, ma è composta da più istruzioni di basso livello (linguaggio macchina) che il computer esegue sequenzialmente.

### Esempio con `counter` (incremento):

1. `registro := counter` carica il valore corrente di `counter` in un registro della CPU.
2. `registro := registro + 1` incrementa il valore nel registro.
3. `contatore := registro` salva il nuovo valore dal registro nella variabile `counter` in memoria.

### Interleaving

Quando un produttore e consumatore operano sul `counter` nello stesso momento, l'esecuzione delle loro istruzioni di basso livello può **intercalarsi** in un ordine che produce un risultato errato.

Esempio:

| Tempo (T) | Processo    | Istruzione Eseguita                       | Risultato (nel registro/contatore) | Stato Errato                     |
|-----------|-------------|-------------------------------------------|------------------------------------|----------------------------------|
| T1        | Produttore  | <code>registro_P := contatore</code> (5)  | registro_P = 5                     |                                  |
| T2        | Produttore  | <code>registro_P := registro_P + 1</code> | registro_P = 6                     |                                  |
| T3        | Consumatore | <code>registro_C := contatore</code> (5)  | registro_C = 5                     | Il Consumatore legge 5 (vecchio) |
| T4        | Consumatore | <code>registro_C := registro_C - 1</code> | registro_C = 4                     |                                  |
| T5        | Produttore  | <code>contatore := registro_P</code> (6)  | contatore = 6                      |                                  |
| T6        | Consumatore | <code>contatore := registro_C</code> (4)  | contatore = 4                      | Risultato Finale: 4              |

Il risultato finale corretto doveva essere 5 (5 elementi iniziali + 1 prodotto - 1 consumato = 5). Invece, l'intercalazione delle istruzioni ha portato a un valore finale di 4 (o 6, a seconda di quale processo finisce per sovrascrivere l'altro).

### Soluzione: Sincronizzazione dei processi

- **Definizione:** la situazione in cui il risultato di un accesso concorrente ai dati dipende dall'ordine specifico in cui i processi accedono è chiamato **Condizione di Gara** (*Race Condition*).
- **Obiettivo:** per evitare le condizioni di gara, è necessario assicurare che un solo processo alla volta possa eseguire la parte critica del codice (ovvero, le istruzioni che modificano i dati condivisi).
- **Necessità:** questa garanzia richiede una forma di sincronizzazione dei processi.

La sincronizzazione è fondamentale nei SO e nelle applicazioni multithread per garantire l'integrità e la coerenza dei dati condivisi.

## Il problema della sezione critica

Il problema della sezione critica è il meccanismo fondamentale da risolvere per garantire che i processi che condividono dati non si interferiscano a vicenda.

Per risolvere il problema, ogni processo viene idealmente suddiviso in quattro segmenti:

- **Sezione Critica (Critical Section):** il segmento di codice in cui il processo accede e modifica variabili comuni (dati condivisi, tabelle, file). Questa è l'*area sensibile* che deve essere protetta.
- **Sezione d'Ingresso (Entry Section):** la sezione di codice che implementa la richiesta del processo di entrare nella Sezione Critica.
- **Sezione d'Uscita (Exit Section):** la sezione di codice che viene eseguita dopo che il processo ha terminato di usare i dati condivisi. Serve a notificare gli altri processi che possono entrare.
- **Sezione non critica:** tutto il codice rimanente, dove il processo non interagisce con i dati condivisi.

Qualsiasi soluzione proposta per risolvere il problema della sezione critica (ovvero, qualsiasi protocollo di ingresso/uscita) deve soddisfare rigorosamente i seguenti tre requisiti:

1. **Mutua esclusione (Mutual Exclusion):** questo è il requisito principale. Se un processo  $P_i$  è nella sua sezione critica, nessun altro processo può essere nella propria sezione critica.
2. **Progresso (progress):** se nessun processo è nella sezione critica e almeno un processo vuole entrare, la decisione su quale processo far entrare non deve essere rimandata indefinitamente e deve coinvolgere solo i processi che vogliono effettivamente entrare. Evita situazioni di deadlock (stallo).
3. **Attesa limitata (bounded waiting):** una volta che un processo  $P_i$  ha espresso l'intenzione di entrare, deve esserci un limite massimo al numero di volte in cui è concesso ad altri processi di entrare nelle loro sezioni critiche prima che  $P_i$  ottenga il permesso. Questo previene la starvation (inedia).

## Implicazioni nel kernel e soluzioni

Il problema non è confinato alle applicazioni utente, ma è cruciale all'interno del kernel, in quanto gestisce tutte le strutture dati condivise del sistema (liste di file aperti, gestione della memoria, assegnazione degli ID di processo come `next_available_pid`).

In un ambiente con un solo core, una soluzione semplice è disabilitare gli interrupt del sistema mentre un processo è nella sua sezione critica.

- Disabilitando gli interrupt, il kernel non può interrompere il processo e passare a un altro (non può esserci prelazione). L'intera sequenza di istruzioni viene eseguita senza interruzioni.
- **Limitazione:** questa soluzione è inefficace (e lenta) nei sistemi multiprocessore (SMP) perché disabilitare gli interrupt richiederebbe di inviare un messaggio a tutti i core, rallentando drasticamente il sistema.

### Strategie del kernel: prelazione

La gestione della sezione critica all'interno del kernel stesso dipende dal suo tipo:

| Tipo di Kernel                                             | Descrizione                                                                                                                                                           | Pro / Contro                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Kernel Senza Diritto di Prelazione (Non-preemptive)</b> | Un processo che esegue codice in modalità kernel <b>non può essere interrotto</b> fino a quando non esce volontariamente dalla modalità kernel (si blocca o termina). | <b>Pro:</b> È naturalmente <b>immune</b> dalle <i>race condition</i> sulle strutture dati del kernel, dato che solo un processo alla volta può usarle.<br><b>Contro:</b> Risposta lenta ( <i>poor latency</i> ) se il codice kernel richiede molto tempo.                                                                                                                                            |
| <b>Kernel Con Diritto di Prelazione (Preemptive)</b>       | Un processo in esecuzione in modalità kernel <b>può essere interrotto</b> ( <i>sottoposto a prelazione</i> ).                                                         | <b>Pro:</b> <b>Maggiore reattività</b> ( <i>responsiveness</i> ) e migliore gestione per i sistemi <b>real-time</b> (a tempo reale).<br><b>Contro:</b> Molto più complesso da progettare, specialmente su architetture SMP, poiché due processi possono eseguire codice kernel contemporaneamente su core diversi, rendendo necessarie soluzioni di sincronizzazione avanzate (come i <i>lock</i> ). |

## Soluzione di Peterson (per due processi)

La soluzione di Peterson è un protocollo che garantisce i tre requisiti di sincronizzazione per due processi  $P_0$  e  $P_1$ .

### 1. Variabili condivise

L'algoritmo utilizza due strutture dati condivise:

- a. `int turn;` : variabile intera che indica di chi è il turno di accedere alla sezione critica. Se `turn == i`, il processo  $P_i$  è autorizzato.

- b. `boolean flag[2]`: un array booleano che indica se un processo è pronto a entrare nella sezione critica. Se `flag[i]` è `true`, il processo  $P_i$  vuole entrare.

## 2. Struttura del processo

La struttura generale di un processo  $P_i$  (dove  $j$  è l'altro processo,  $j = 1 - i$ ) nella soluzione di peterson è la seguente:

```
while (true) {
    /* SEZIONE D'INGRESSO */
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ; /* non fa niente, attesa attiva (busy waiting) */

    /* SEZIONE CRITICA */
    /* Codice che accede ai dati condivisi */

    /* SEZIONE D'USCITA */
    flag[i] = false;

    /* SEZIONE NON CRITICA */
    /* Codice rimanente */
}
```

### Sezione d'ingresso:

- $P_i$  segnala che vuole entrare: `flag[i] = true;`
- $P_i$  dà la "precedenza" all'altro processo: `turn = j;`
- $P_i$  si blocca se l'altro processo vuole entrare e ha la precedenza:

```
while (flag[j] && turn == j)
    ; /* non fa niente */
```

### Sezione critica:

- (esecuzione del codice protetto)

### Sezione d'uscita:

- $P_i$  dichiara di aver finito: `flag[i] = false;`

## 3. Mutua esclusione

La mutua esclusione è garantita dalla condizione del ciclo `while`:

- un processo  $P_i$  può entrare nella sezione critica solo se la condizione `flag[j] && turn == j` è falsa.

### Cosa succede se entrambi tentano di entrare contemporaneamente?

1. Entrambi impostano il loro `flag` a `true`.
2. Entrambi scrivono su `turn`. Solo l'ultima scrittura "vince" (es. `turn` diventa `j`).
3. Supponiamo che `turn` sia rimasto `j`.
  - $P_i$  esegue il `while` e trova: `flag[j]` (true) e `turn == j` (true)  $\rightarrow P_i$  si blocca.
  - $P_j$  esegue il `while` e trova `flag[i]` (true) ma `turn == j` (true)  $\rightarrow P_j$  entra.
4. Il processo che ha perso l'ultima scrittura su `turn` viene bloccato, garantendo che uno solo alla volta entri.

Inoltre, l'algoritmo soddisfa anche il progresso (se uno non vuole entrare, l'altro può) e l'attesa limitata (un processo può aspettare al massimo una singola esecuzione dell'altro, poiché k'altro al termine resetta il proprio `flag`).

La soluzione di Peterson è corretta teoricamente assumendo che le istruzioni vengano eseguite nell'ordine esatto in cui sono scritte. Questo non è garantito sui moderni elaboratori a causa dell'ottimizzazione dei processori e dei compilatori.

### 1. Riordino delle istruzioni (Instruction Reordering)

I moderni sistemi riorganizzano l'ordine delle istruzioni di lettura e scrittura in memoria per migliorare le prestazioni, a meno che non ci siano dipendenze dirette tra le variabili.

Esempio:

Dati condivisi:

```
boolean flag = false;
int x = 0;
```

Thread 1:

```
while (!flag)
    ;
print x;
```

Thread 2:

```
x = 100;
flag = true;
```

- **Comportamento Atteso:** Il Thread 1 aspetta che `flag` sia `true`, poi stampa `x = 100`.
- **Comportamento Dopo Riordino:** Il processore può riordinare le istruzioni di Thread 2 in:
  1. `flag = true;` (Il Thread 1 vede subito `flag = true` ed entra)
  2. `x = 100;` (La scrittura di `x` avviene **dopo** che il Thread 1 ha letto `x`)
- **Risultato Errato:** Il Thread 1 stampa `x = 0` (il valore iniziale), anche se la logica software prevedeva 100.

## 2. Fallimento di Peterson con riordino

Se gli assegnamenti nella sezione d'ingresso di Peterson (`flag[i] = true;` e `turn = j;`) vengono riordinati, i meccanismi di sicurezza dell'algoritmo vengono violati, e la mutua esclusione può fallire.

Entrambi i processi potrebbero ritrovarsi a credere di aver dato la precedenza all'altro, mentre in realtà stanno per entrare entrambi.

## Supporto hardware per la sincronizzazione

Le soluzioni puramente software falliscono sui moderni sistemi a causa del riordino delle istruzioni e dei diversi modelli di memoria dei processori. Per questo, si ricorre a istruzioni hardware specifiche che garantiscono operazioni atomiche (non interrompibili).

### 1. Barriera di memoria (Memory Barriers)

Le barriere di memoria sono istruzioni che forzano l'ordine di esecuzione delle operazioni di memoria e ne garantiscono la visibilità tra i diversi processori.

#### Modelli di memoria

- **Fortemente ordinato:** una modifica alla memoria è immediatamente visibile a tutti i processori (garanzia di coerenza).
- **Debolmente ordinato:** le modifiche non sono immediatamente visibili, permettendo riordini per ottimizzare le prestazioni. (Richiede l'uso esplicito di barriere).

#### Funzione della barriera

Quando viene eseguita una barriera di memoria (`memory_barrier();`), il sistema assicura che tutte le istruzioni di **Load e Store** precedenti siano **completate** e visibili a tutti i core **prima** che vengano eseguite le istruzioni di Load e Store successive.

### 2. Istruzioni hardware atomiche

Molte architetture offrono istruzioni speciali che leggono e modificano una parola di memoria come **unità non interrompibile (atomica)**. Queste istruzioni sono le primitive fondamentali per costruire tutti i meccanismi di sincronizzazione di alto livello.

#### a. `test_and_set()`

Questa istruzione (prova e imposta) esegue atomicamente due azioni su una variabile:

- **legge** il valore corrente della variabile (target);
- **imposta** la variabile a `true`.

#### Definizione:

```
boolean test_and_set (boolean *target) {
    boolean rv = *target; // 1. legge il valore originale
    *target = true; // 2. imposta il valore a 'true'
```

```
    return rv; // 3. restituisce il valore letto
}
```

### Uso per la mutua esclusione

Si usa una variabile globale `lock` (inizializzata a `false`).

```
do {
    while (test_and_lock(&lock))
        ; /* do nothing */
    /* critical section */

    lock = false;
    /* remainder section */
} while (true);
```

La funzione `test_and_lock(&lock)` viene eseguita atomicamente:

- legge il valore corrente di `lock`
- imposta `lock` a `true`
- restituisce il valore che ha letto (il vecchio valore di `lock`)

Se il lock è libero (`lock == false`)

- la funzione legge `false` e lo imposta subito a `true`
- l'istruzione restituisce `false`
- il ciclo `while(false)` fallisce e il processo entra nella sezione critica

Se il lock è già acquisito (`lock == true`)

- la funzione legge `true` e lo imposta di nuovo a `true`
- l'istruzione restituisce `true`
- il ciclo `while(true)` continua e il processo resta bloccato in attesa attiva (*busy waiting*), provando continuamente ad acquisire il lock.

Una volta terminata la sezione critica, il processo imposta `lock = false`.

Questo permette al primo processo bloccato in attesa di eseguire la successiva `test_and_lock`, trovare `false`, uscire dal ciclo `while` e procedere all'ingresso.

### b. `compare_and_swap()`

L'istruzione (confronta e scambia) è più versatile e opera su tre valori:

- un indirizzo di memoria (`*value`),
- un valore atteso (`expected`),
- un nuovo valore (`new_value`).

### Definizione:

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp; // restituisce sempre il valore originale
}
```

### Uso per la mutua esclusione

Si usa una variabile globale `lock` (inizializzata a `0`).

```
while (true) {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */
    /* critical section */
}
```

```

    /* critical section */

    lock = 0;

    /*remainder section */
}

```

`compare_and_swap(&lock, 0 1)` è una singola operazione atomica che esegue tre azioni:

- **legge** il valore corrente di `lock`
- **confronta** se il valore di `lock` è uguale al valore atteso (`0`), lo **scambia** impostandolo a `1`
- **restituisce** il valore originale di `lock`

#### Caso 1: Lock libero (`lock == 0`)

- il primo processo che esegue la `cas` trova `lock` uguale a `0`
- la `cas` imposta `lock` a `1` (occupato) e restituisce `0` (il valore originale)
- il ciclo `while (0 != 0)` fallisce e il processo entra nella sezione critica

#### Caso 2: Lock occupato (`lock == 1`)

- I processi successivi che eseguono la `cas` trovano `lock` uguale a `1`
- la condizione di confronto fallisce, quindi la `cas` non modifica `lock`
- la `cas` restituisce `1` (valore originale)
- il ciclo `while (1 != 0)` ha successo e il processo resta bloccato in attesa attiva, provando continuamente ad acquisire il lock.

Una volta che il processo ha terminato il lavoro nella sezione critica, imposta semplicemente `lock = 0`.

Questo libera la risorsa, permettendo al primo processo in attesa di eseguire la `cas` con successo e di acquisire il lock.

### Attesa limitata con `compare_and_swap()`

Sia `test_and_set()` che `compare_and_swap()` risolvono la mutua esclusione. Tuttavia, non risolvono il requisito di attesa limitata (un processo potrebbe aspettare per sempre, *starvation*).

Per risolvere l'attesa limitata, si usa una variante più complessa di `cas` che introduce un array di attese (`waiting[n]`) per gestire l'ordine di ingresso.

1. **Ingresso:** un processo si mette in attesa (`waiting[i] = true;`) e tenta di acquisire il lock. Se fallisce, rimane bloccato solo finché `waiting[i]` non viene impostato a `false` dall'altro processo in uscita.
2. **Uscita:** quando un processo esce dalla sezione critica, esso scandisce ciclicamente l'array `waiting`. Trova il primo processo in attesa (`waiting[j] == true`) e imposta solo a quel processo `waiting[j] = false;`.
3. **Garantito:** questo garantisce che ogni processo in attesa ottenga l'ingresso entro un numero limitato ( $n - 1$ ) di cicli, soddisfacendo l'attesa limitata.

```

while (true) {
    waiting[i] = true;
    key = 1;

    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1); // tenta di prendere il lock
    waiting[i] = false; // il lock è stato acquisito, esce dalla coda di attesa e entra nella sezione critica

    /* critical section */

    /* attesa limitata: passa il permesso al prossimo processo in attesa
    invece di liberare il lock e rischiare che un processo appena arrivato lo prenda per primo*/
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n; // cerca il primo processo in attesa
}

```

```

    if (j == i)
        lock = 0; //nessuno sta aspettando => il lock viene rilasciato
    else
        waiting[j] = false; //viene trovato un successore => il lock non viene rilasciato dando il permesso al processo j di eseguire

    /* remainder section */
}

```

## Lock Mutex

I lock mutex (da *MUTual EXclusion*) sono il meccanismo di sincronizzazione più semplice (a livello software) fornito dai SO per proteggere le sezioni critiche e prevenire la race condition.

Un lock mutex ha una variabile booleana, `available`, che indica se il lock è disponibile (`true`) o meno (`false`).

Un processo usa due funzioni atomiche:

- `acquire()`: il processo acquisisce il lock prima di entrare nella sezione critica.
  - Se il lock è disponibile, lo prende (`available = false`) ed entra.
  - Se non è disponibile, il processo attende.
- `release()`: il processo rilascia il lock quando esce dalla sezione critica (`available = true`).

### Il problema dell'attesa attiva

L'implementazione semplice di `acquire()` presenta un problema: l'attesa attiva.

```

acquire() {
    while (!available)
        ; /* busy waiting */
    available = false;
}

```

Le chiamate a `acquire()` e `release()` devono essere eseguite atomicamente, perciò i lock mutex sono spesso realizzati utilizzando dei meccanismi hardware.

Se il lock non è disponibile, il processo rimane in un ciclo continuo, sprecando cicli di CPU.

- **Spinlock**: è un lock che utilizza l'attesa attiva. Sebbene inefficiente nei sistemi monoprocesso, gli spinlock possono essere utili nei sistemi multiprocessore se tenuti per tempi molto brevi, perché evitano il costo del cambio di contesto.

## Semafori

I semafori sono uno strumento di sincronizzazione più potente e robusto dei lock mutex, sebbene un semaforo binario (che può assumere solo 0 o 1) si comporti in modo simile a un lock mutex.

Un semaforo  $S$  è una variabile intera a cui si può accedere solo tramite due operazioni atomiche e indivisibili:

1. `wait(S)`: diminuisce il valore di  $S$ . Se  $S$  diventa non positivo (cioè  $\leq 0$ ), il processo deve attendere.

```

wait (S) {
    while(S <= 0)
        ; //busy waiting
    S--;
}

```

2. `signal(S)`: aumenta il valore di  $S$ .

```

signal(S) {
    S++;
}

```

I semafori si distinguono tra:

- **Semafori binari**: hanno un valore limitato tra 0 e 1.

- **Semafori contatori:** il semaforo è inizialmente impostato al numero di risorse disponibili, i processi che vogliono utilizzare una risorsa invocano la `wait()` decrementando il valore. Quando il semaforo vale 0, tutte le risorse sono occupate e i processi che vogliono utilizzarlo restano bloccati finché il valore non torna positivo.

### Implementazione dei semafori

L'implementazione dei semafori ha come obiettivo quello di eliminare l'attesa attiva.

Per farlo, ogni semaforo  $S$  deve contenere due elementi:

1. `value`: la variabile intera del semaforo.
2. `list`: una lista di processi in attesa (o bloccati) su quel semaforo.

Le operazioni atomiche `wait(S)` e `signal(S)` vengono ridefinite per includere la gestione di questa lista.

### **Operazione `wait(S)` (blocco)**

Quando un processo chiama `wait(S)`:

1. **Decremento:** il valore del semaforo viene decrementato: `S.value--`.
2. **Verifica:**
  - Se `S.value` è ancora `>= 0`: significa che ci sono risorse disponibili (o che la risorsa non è ancora completamente esaurita, se si tratta di un semaforo contatore). Il processo continua la sua esecuzione ed entra nella sezione critica.
  - Se `S.value` è `< 0`: significa che non ci sono risorse disponibili e il processo deve attendere.
    - Il processo viene bloccato (passa da *running* a *waiting*).
    - Il processo viene aggiunto alla lista di attesa (`S.list`).
    - Il controllo della CPU passa allo scheduler, che esegue un altro processo.

Quando `S.value` è negativo, il suo valore assoluto indica esattamente quanti processi sono attualmente in attesa nella lista del semaforo.

### **Operazione `signal(S)` (risveglio)**

Quando un processo chiama `signal(S)`:

1. **Incremento:** il valore del semaforo viene incrementato: `S.value++`.
2. **Verifica:**
  - Se `S.value` è `≤ 0`: significa che c'erano processi in attesa (poiché il valore era precedentemente negativo).
    - Un processo viene rimosso dalla lista di attesa.
    - Il processo rimosso viene risvegliato (passa da *waiting* a *ready*) e messo nella coda dei processi pronti per essere eseguito.
  - Se `S.value` è `> 0`: significa che non ci sono processi in attesa. La risorsa è semplicemente stata rilasciata e resa disponibile per il prossimo processo che chiamerà `wait()`.

## Monitor

Mentre i semafori sono efficaci, il loro uso errato può portare a errori di sincronizzazione difficili da debuggare.

Il monitor è un costrutto di sincronizzazione di livello superiore, progettato per risolvere questi problemi, rendendo la sincronizzazione più semplice e sicura.

Un monitor è un tipo di dato astratto che contiene:

- dichiarazioni di **variabili condivise**;
- definizioni di **procedure/funzioni** che operano su tali variabili.

La caratteristica fondamentale di un monitor è la **mutua esclusione automatica**:

- **Un solo processo alla volta:** il monitor garantisce che in un dato momento, al massimo un processo possa essere attivo all'interno di qualsiasi delle sue procedure (funzioni).
- **Accesso controllato:** se un processo tenta di chiamare una procedura del monitor mentre un altro è già attivo al suo interno, il nuovo processo viene bloccato e messo in una coda di ingresso associata al monitor.
- **Niente `acquire()` / `release()`:** il programmatore non deve preoccuparsi di inserire manualmente le chiamate `wait()` o `signal()` per proteggere i dati; la protezione è gestita automaticamente dal compilatore / runtime del monitor.



### Sincronizzazione aggiuntiva: variabili condition

Per schemi di sincronizzazione più complessi (dove un processo potrebbe aver bisogno di attendere una condizione), il monitor usa le variabili speciali di tipo `condition` ( $x$ ).

- `x.wait()`: il processo che invoca questa operazione sospende la propria esecuzione (e implicitamente rilascia il monitor), in attesa che la condizione  $x$  si verifichi.
- `x.signal()`: risveglia esattamente un processo sospeso sulla variabile condizione  $x$ . Se non ci sono processi in attesa, `signal()` non ha alcun effetto.

#### ▼ Flusso di esecuzione (esempio)

Immagina un Monitor chiamato `BufferMonitor` con una procedura `inserisci()`.

##### 1. Ingresso e Mutua Esclusione

- **Processo A chiama `BufferMonitor.inserisci()`:**
  1. Il Monitor **acquisisce il lock** (mutua esclusione).
  2. L'esecuzione entra in `inserisci()`.
- **Processo B chiama `BufferMonitor.inserisci()` mentre A è attivo:**
  1. Il Monitor tenta di acquisire il lock.
  2. Il lock è occupato.
  3. Il Processo B viene messo in una **Coda d'Ingresso e bloccato** (stato *Waiting*). B aspetta che A rilasci il lock.

##### 2. Sospensione (La `wait()` della Condition Variable)

Ora, supponiamo che A, all'interno di `inserisci()`, trovi che il buffer è **pieno** e deve attendere. A esegue:

```
if (buffer_pieno) {
    buffer_non_pieno.wait();
}
```

La `wait()` ha un doppio effetto critico:

1. **Sblocco degli Altri:** A **rilascia immediatamente il lock del Monitor**. Questo è fondamentale, altrimenti nessun altro processo (incluso il Consumatore) potrebbe mai entrare per cambiare lo stato del buffer.
2. **Blocco di A:** A viene messo in una **coda di attesa** specifica per la condizione `buffer_non_pieno` e passa allo stato *Waiting*.

Il Monitor è ora libero. Lo scheduler può prendere B dalla Coda d'Ingresso, fargli acquisire il lock e iniziare l'esecuzione.

##### 3. Risveglio (La `signal()` della Condition Variable)

Supponiamo che il Consumatore  $P_k$  entri nel Monitor e crei spazio. Alla fine della sua procedura,  $P_k$  esegue:

```
buffer_non_pieno.signal();
```

- **Risveglio di A:** `signal()` prende il Processo A dalla coda di attesa e lo sposta in una coda di processi pronti a rientrare nel Monitor.

##### 4. Ripresa

- Quando  $P_k$  esce dalla sua procedura, **rilascia il lock del Monitor**.
- Il Processo A, ora risvegliato, **ottiene il lock** ed entra nel Monitor.
- A riprende l'esecuzione **esattamente dopo la sua chiamata `wait()`**, cioè ricontrolla che il buffer non sia più pieno e poi procede.

Il Monitor gestisce tutto l'incantesimo del *blocco*, del *rilascio del lock* e del *risveglio* in un'unica unità astratta. Al programmatore restano solo le semplici regole di `wait()` quando non può procedere e `signal()` quando ha cambiato lo stato.

### Realizzazione di un monitor con i semafori

A ogni monitor si associa un semaforo `mutex`, per ogni istanza del monitor abbiamo bisogno delle variabili:

- `semaphore mutex`: inizializzato a 1. Questo semaforo è la chiave per la mutua esclusione. Garantisce che solo un processo alla volta possa eseguire il codice di una qualsiasi funzione all'interno del monitor.
- `semaphore next`: inizializzato a 0. Questo semaforo è utilizzato per l'implementazione della `signal_and_wait()`. Controlla il processo che esegue `signal` in modo che possa mettersi in pausa per far rientrare immediatamente il processo risvegliato.
- `int next_count`: inizializzato a 0. Contatore dei processi che sono stati messi in attesa sul semaforo `next`.

Inoltre, per ogni variabile condition  $x$  (es. `buffer_not_empty`), sono necessarie:

- `semaphore x_sem`: inizializzato a 0. Questo semaforo viene utilizzato per sospendere i processi che eseguono `x.wait()`.
- `int x_count`: inizializzato a 0. Contatore dei processi attualmente in attesa sulla condizione  $x$ .

Ogni funzione  $F$  all'interno del monitor è sostituita da un blocco di codice che gestisce l'ingresso e l'uscita, garantendo la mutua esclusione.

```
wait(mutex); // Acquisizione del lock per la Mutua Esclusione
...
Body of F; // Esecuzione della logica originale
...
if (next_count > 0)
    signal(next); // Se ci sono processi bloccati dalla signal, li sblocca (Signal-and-Wait)
else
    signal(mutex); // Altrimenti, rilascia il lock per il prossimo processo in coda d'ingresso
```

- **Ingresso** (`wait(mutex)`): il processo prende il lock, assicurando che sia l'unico attivo nel monitor.
- **Uscita** (`signal` condizionale): quando il processo esce, controlla se ci sono processi in attesa sul semaforo `next` (cioè processi che hanno eseguito una `signal` e si sono messi in attesa).
  - Se `next_count > 0`: risveglia uno di questi processi in attesa su `next` (passandogli il controllo).
  - Altrimenti rilascia il `mutex` del monitor per il prossimo processo nella coda d'ingresso.

Un processo che si mette in attesa deve rilasciare il lock del monitor prima di sospendersi, `x.wait()`:

```
x_count++; // 1. Registra l'attesa sulla condizione x

if (next_count > 0)
    signal(next); // 2. Se c'è chi attende un permesso (signal), gli passa il Monitor
else
    signal(mutex); // 3. Altrimenti, rilascia il Monitor per la coda d'ingresso

wait(x_sem); // 4. Si sospende sul semaforo di condizione

x_count--; // 5. Al risveglio, deregistra l'attesa
```

1. `x_count++` aumenta il contatore per sapere quanti processi sono in attesa sulla condizione  $x$ .
2. `if/else signal`: esegue la logica di uscita del monitor.
3. `wait(x_sem)`: il processo si sospende su un semaforo che è inizializzato a 0. Qui si blocca finché un altro processo non esegue la `x.signal()`.
4. `x_count--`: una volta risvegliato (dopo la `signal`), decrementa il contatore.

La `x.signal()` è responsabile del risveglio di un processo, se ce n'è uno in attesa:

```
if (x_count > 0) { // 1. Controlla se c'è qualcuno in attesa su x
    next_count++; // 2. Aumenta il contatore "next": il processo chiamante si aspetta di attendere
    signal(x_sem); // 3. Risveglia un processo in attesa su x
    wait(next); // 4. Si sospende sul semaforo "next" per far eseguire il processo risvegliato
    next_count--; // 5. Al suo risveglio, il processo chiamante è fuori dal Monitor
}
```

1. `if (x_count > 0)`: controlla se ci sono processi bloccati sulla condizione  $x$ .

2. `next_count++` e `wait(next)` : implementano la `signal_and_wait` . Il processo chiamante  $Q$  (che esegue la `signal` ) si sospende immediatamente su `next` per cedere la priorità al processo  $P$  appena risvegliato.
3. `signal(x_sem)` : risveglia il processo  $P$  che era in attesa sulla `x.wait()` .
4. `next_count--` : dopo che  $P$  ha finito di eseguire, risveglia  $Q$  che decrementa il contatore `next` .

## Implementazione delle Variabili condition

Le variabili condition sono fondamentali per la sincronizzazione all'interno del monitor. Tuttavia, la loro implementazione crea un problema di concorrenza tra i due processi coinvolti:

- Processo  $P$  (che esegue `x.signal()` ) : è attualmente attivo nel monitor.
- Processo  $Q$  (risvegliato da `x.signal()` ) : è pronto per entrare nel monitor.

Entrambi i processi desiderano essere attivi nel monitor contemporaneamente, il che viola il principio di mutua esclusione del monitor stesso.

Per gestire questo conflitto:

### 1. Signal-and-Wait

- Il processo che esegue `signal()` ( $P$ ) si sospende immediatamente, cedendo il controllo del monitor al processo risvegliato  $Q$ .
- $Q$  entra ed esegue. Quando  $Q$  esce dal monitor, risveglia  $P$ .
- Garantisce che, quando  $Q$  riprende, la condizione che ha causato il risveglio sia ancora vera, poiché  $P$  non ha avuto modo di alterare lo stato dopo la `signal()` .

### 2. Signal-and-Continue

- Il processo che esegue `signal()` ( $P$ ) continua a eseguire la sua sezione critica all'interno del monitor. Il processo risvegliato  $Q$  deve aspettare che  $P$  esca del tutto dal monitor.
- $Q$  viene posto in una coda di urgenza o nella coda d'ingresso e attende che  $P$  abbia finito.
- Quando  $Q$  finalmente rientra, non c'è più la garanzia che la condizione per cui era svegliato sia ancora valida. Questo schema, infatti, richiede che il processo  $Q$  ricontrrolli sempre la condizione con un ciclo `while` dopo il risveglio.

## Liveness

Il concetto di liveness si riferisce alle proprietà che garantiscono che i processi facciano progresso durante l'esecuzione. La "mancanza di liveness" (*liveness failure*) si verifica quando un processo non riesce a progredire.

## Deadlock (stallo)

Il deadlock si verifica quando due o più processi sono in attesa indefinita di una condizione che può essere soddisfatta solo da uno degli altri processi in attesa.

- *Esempio*: due processi  $P_1$  e  $P_2$ .  $P_1$  ha il lock  $L_1$  e vuole  $L_2$ .  $P_2$  ha il lock  $L_2$  e vuole  $L_1$ . Nessuno può procedere, sono interbloccati.

## Starvation (Inedia)

La starvation si verifica quando un processo non è mai in grado di accedere a una risorsa (o alla sezione critica) a causa di una gestione inefficiente o sbilanciata della sincronizzazione.

- *Esempio*: un sistema di gestione delle risorse basato su priorità, dove un processo a bassa priorità non ottiene mai la risorsa perché ci sono sempre processi a priorità alta che la richiedono.

## Inversione di priorità

L'inversione di priorità è un problema specifico che si verifica nei sistemi real-time o basati su priorità:

1. Un processo a **bassa priorità** ( $L$ ) acquisisce un lock (risorsa  $R$ ).
2. Un processo a **alta priorità** ( $H$ ) ha bisogno della risorsa  $R$  e si blocca, aspettando  $L$ .
3. Un processo a **media priorità** ( $M$ ) diventa eseguibile e preleva  $L$ .
4.  $L$  non può eseguire e quindi non può rilasciare  $R$ .

5. Il processo ad alta priorità  $H$  è costretto ad aspettare che il processo a media priorità  $M$  finisca, invertendo l'ordine di priorità atteso.

Soluzione (**Ereditarietà di Priorità**): per risolvere l'inversione di priorità, si usa il protocollo di ereditarietà di priorità. Quando un processo a bassa priorità detiene una risorsa necessaria a un processo ad alta priorità, il processo a bassa priorità eredita temporaneamente la priorità più alta fino a quando non rilascia la risorsa.